



Sandya Mannarswamy

CODE SPORT

Welcome to CodeSport! In this month's column, we turn our attention to software coding errors, which lead to buggy code. We will discuss different types of software bugs caused by software coding errors, and the various static and runtime bug-detection tools used to catch these software errors.

Programming/coding errors are a major cause of the software defects encountered after the software has been commercially released and is in use. Late detection of such coding errors after they have manifested themselves to end-users impacts software maintenance costs. Detecting a programmer's error during development itself leads to lowered software maintenance costs, and helps improve the customer experience of the released software. Therefore, detecting bugs early in the development cycle is important. There are certain kinds of common coding errors, which occur often, and a developer needs to keep a vigilant eye to avoid these. Let's look at some of the more common variety.

Common coding errors

Null pointers

Typically, programmers forget to check whether a pointer can be *NULL* before dereferencing it. This is known as a null pointer dereferencing error. Consider the following code snippet:

```
int* bar(int cond)
{
    int* alloc = 0;
    if (cond == 0)
        return 0;
    else
    {
        alloc = (int*) malloc(sizeof(int) * cond);
        if (alloc == 0)
```

```
        return 0;
    }
    return alloc;
}

int foo(int a)
{
    int *p = bar(a);

    int result = *p;
    return result;
}
```

In the function 'foo', the variable 'result' is set to the value obtained by dereferencing the pointer 'p'. However, there is no check to see if 'p' is non-null before dereferencing it. This is a bad coding error, since 'p' is obtained as the return value from the function 'bar', and can be zero in certain cases. This error, wherein the programmer does not check for a pointer being non-null before dereferencing it, is known as a 'potential null pointer dereference error'. This is a difficult bug to catch at run-time, since depending on the inputs, 'p' can be null or non-null. If the set of inputs we are using to test the code do not result in 'p' being null, then this issue cannot be caught by testing. However, most static source-code-analysis tools can detect this issue.

Memory access/usage errors

Another area where programmers frequently make coding errors is in dealing with dynamically allocated memory. There are two classes of errors